

1. ГЛАВА 1. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

1.1. Пример реализации: Web Applications for Robots using rosbridge

В 2012 году Jihoon Lee (Brown University) предложил архитектуру, объединяющую возможности операционной системы роботов ROS и технологий веб-программирования: rosbridge, rosj и mjpeg_server — для построения полноценных веб-приложений для роботов [Lee, 2012 TODO] (https://cs.brown.edu/media/filer_public/b0/03/b0037d43-76b6-4729-8c1e-635f120e5192/lee.pdf). Ниже приведено подробное описание этой архитектуры и её практического применения в нескольких сценариях, которые могут служить примером реализации системы удалённого управления роботом через Интернет.

1.1.1. Архитектура системы

Основу предлагаемой архитектуры (Рисунок 1.1) составляют четыре ключевых компонента: ROS, rosbridge, rosj и mjpeg_server.

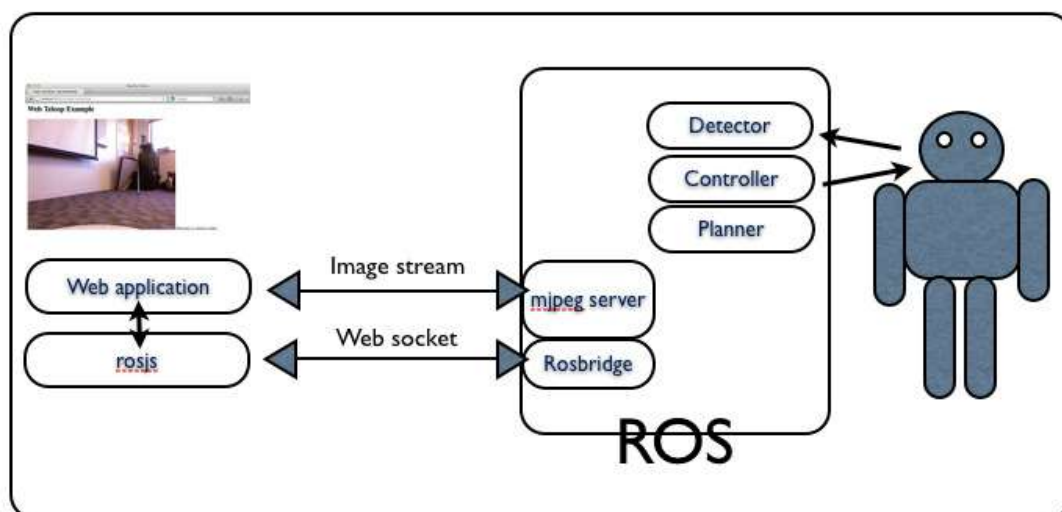


Рисунок 1.1 Архитектура системы rosbridge

1. **ROS (Robot Operating System)** выполняет функции серверной части — обрабатывает команды, управляет аппаратными драйверами и планированием действий, а также публикует данные датчиков и состояния.
2. **rosbridge** служит «мостом» между ROS и внешними клиентами. Он предоставляет унифицированный протокол на основе WebSocket или TCP, сериализуя сообщения ROS в формате JSON. Благодаря этому веб-клиент получает доступ ко всем функциям ROS (топики, сервисы, параметры) без необходимости локальной установки.

3. **rosjs** — это JavaScript-библиотека для браузера, упрощающая работу с rosbridge. Она предоставляет высокоуровневые методы `publish()`, `subscribe()` и `callService()`, позволяющие веб-программисту взаимодействовать с ROS-системой, не вдаваясь в её внутренние механизмы.
4. **mjpeg_server** обеспечивает трансляцию видеопотока от робота через HTTP в формате MJPEG. Поток можно параметризовать по качеству и разрешению, что делает систему гибкой при ограниченной пропускной способности сети.

Такое разделение позволяет создавать интерфейсы управления, полностью работающие в браузере, но взаимодействующие с реальной аппаратурой через ROS.

1.1.2. Протоколы и обмен данными

Коммуникация между веб-клиентом и ROS осуществляется через WebSocket-соединение, установленное между `rosjs` и `rosbridge`. Каждое сообщение формируется в формате JSON, включающем тип операции (`publish`, `subscribe`, `call_service`), имя топика и тело сообщения.

Видеопоток передаётся отдельным HTTP-соединением через `mjpeg_server`, который публикует последовательность JPEG-кадров. Такой подход прост в реализации и совместим со всеми браузерами.

1.1.3. Примеры реализованных веб-приложений

PR2 Castle Builder

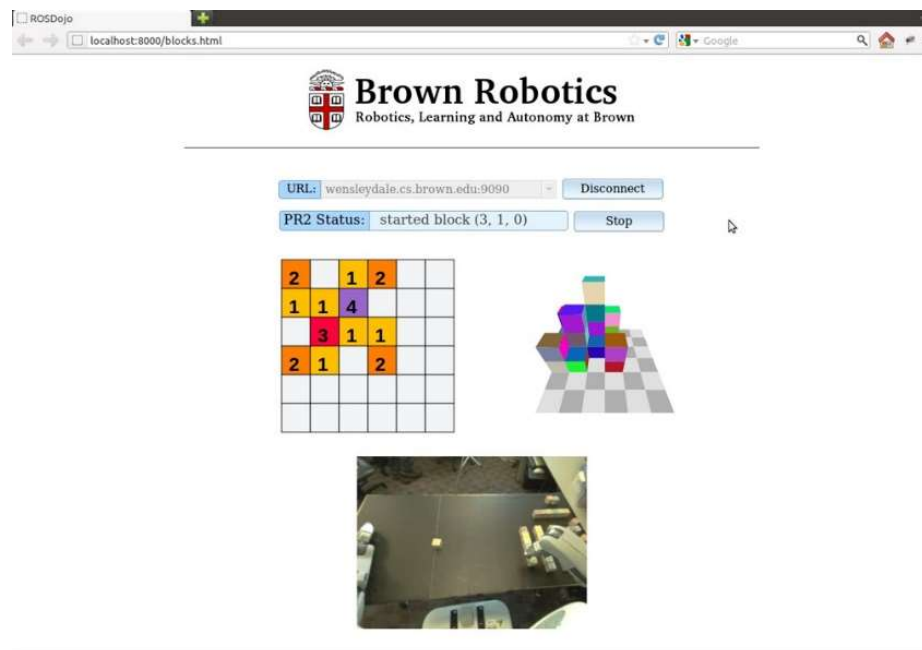


Рисунок 1.2 Веб-интерфейс Castle Builder

Данное приложение позволяет пользователю дистанционно управлять роботом PR2, собирающим «замок» из кубиков.

Работа интерфейса строится следующим образом:

1. Пользователь подключается к `rosbridge` и запускает поток видео от робота.
2. В веб-интерфейсе он проектирует модель замка в 2D-среде (6×6 ячейки), задавая количество блоков в каждой ячейке.
3. Интерфейс генерирует список позиций блоков и отправляет их в ROS-систему, где планировщик формирует последовательность операций «взять-положить».
4. После выполнения всех действий робот публикует сообщение о завершении.

Castle Builder демонстрирует полный цикл взаимодействия — проектирование, отправку команд и контроль результата через веб-интерфейс.

PR2 Commander (естественно-языковое управление)



Рисунок 1.3 Веб-интерфейс PR2 Commander

Второе приложение реализует управление роботом посредством естественного языка.

Веб-интерфейс принимает текстовые или голосовые команды, которые передаются через `rosbridge` в модуль SLU (Spatial Language Understanding). SLU анализирует фразу и возвращает структурированные инструкции, например: «возьми красный куб и поставь его на синий».

Таким образом, веб-интерфейс становится универсальной точкой доступа к семантическим сервисам и позволяет комбинировать голосовое управление с ROS.

PR2 Turntable Manipulator (оператор в контуре управления)

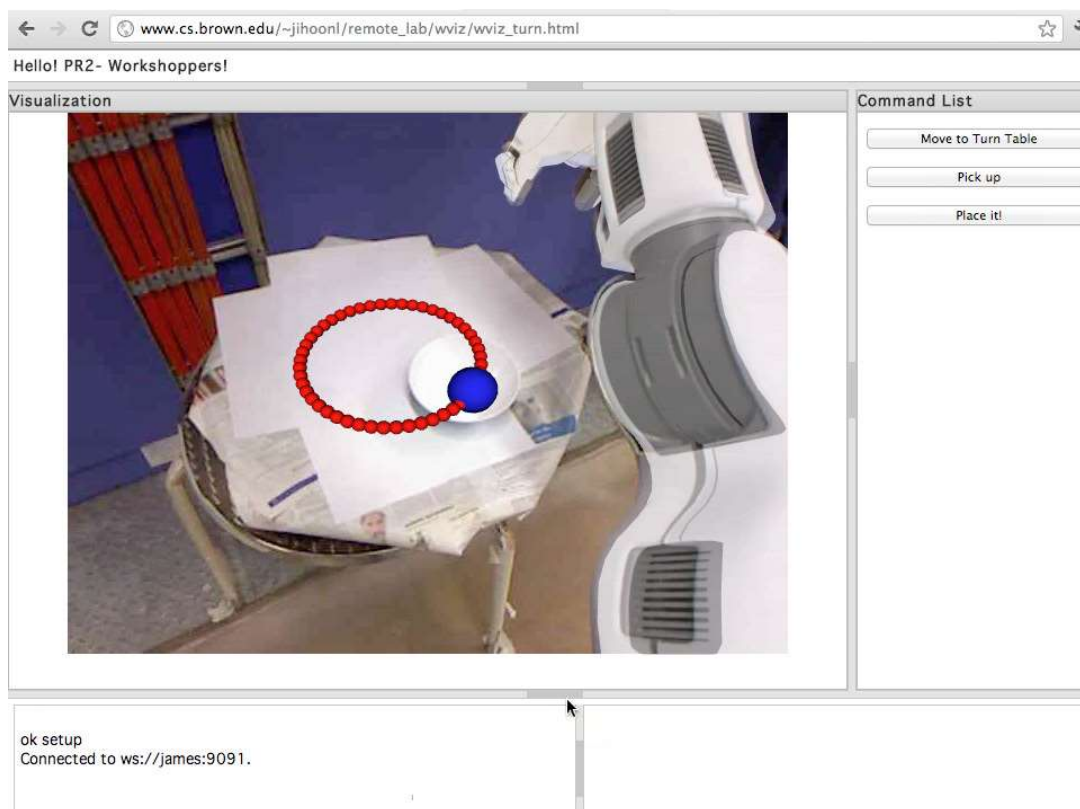


Рисунок 1.4 Интерфейс Turntable Manipulator

Здесь человек выступает активным участником процесса: он наблюдает поток видео, на котором наложена предсказанная траектория движения объекта, и выбирает момент для команды «схватить».

Этот сценарий иллюстрирует концепцию human-in-the-loop — совместное принятие решений человеком и роботом для повышения точности выполнения задач.

1.1.4. Среда ROSProcessingjs

Для упрощения создания визуальных веб-приложений автор предложил среду ROSProcessingjs, объединяющую возможности языка Processing и библиотеки Processing.js.

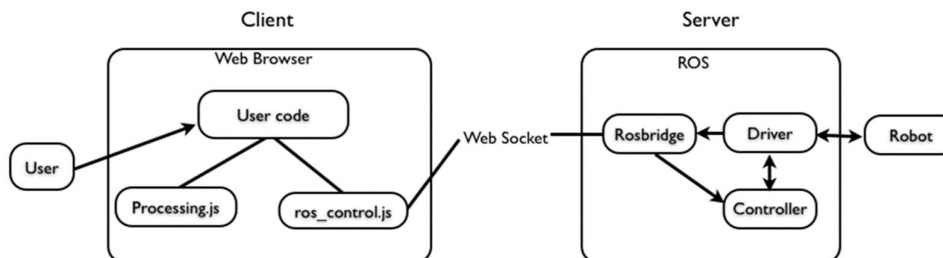


Рисунок 1.5 Схема ROSProcessingj

В этой среде пользователь пишет код на Processing, который автоматически транслируется в JavaScript и выполняется в браузере. Через rosjs код взаимодействует с rosbridge, что делает разработку интуитивной даже для студентов без глубоких знаний JS.

Пример кода для управления движением робота приведён ниже ().

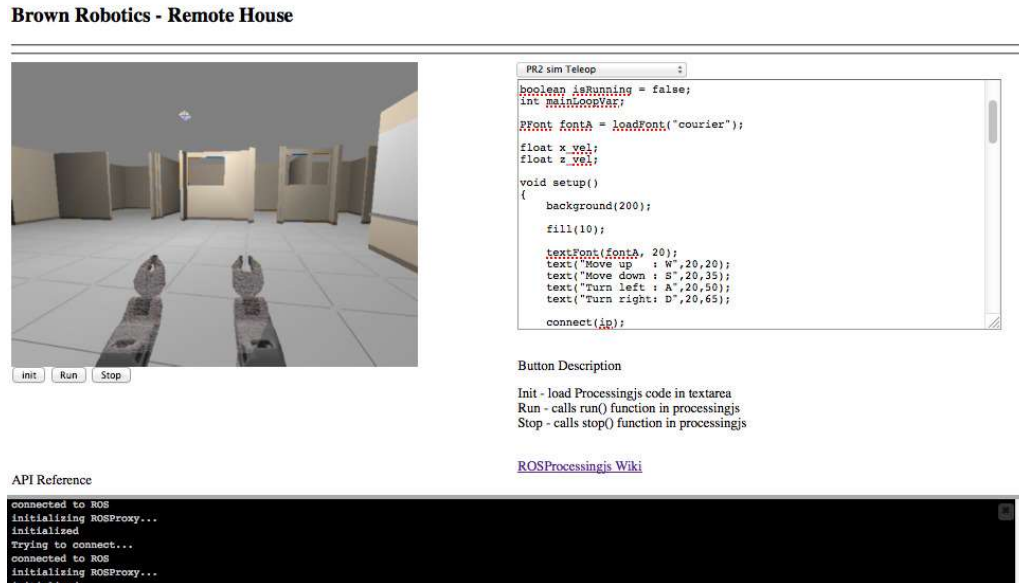


Рисунок 1.6 Фрагмент кода телеприсутствия

Код устанавливает соединение, получает поток изображения и отправляет команды на топик /cmd_vel, реализуя базовое телуправление роботом.

1.1.5. Преимущества и ограничения подхода

Преимущества:

- Кроссплатформенность: доступ к роботу из любого браузера.
- Низкий порог входа для разработчиков UI.
- Возможность быстрого прототипирования и образовательного использования.
- Гибкая архитектура: rosbridge поддерживает разные протоколы и может интегрировать несколько клиентов.

Ограничения:

- В статье не рассмотрены вопросы безопасности при удалённом доступе. Для промышленного или учебного применения требуется аутентификация, контроль прав и изолированная песочница для кода.
- Использование MJPEG удобно, но не обеспечивает минимальную задержку. Для современных систем предпочтителен WebRTC.

- Система не описывает многопользовательский доступ и масштабирование; эти аспекты следует доработать в будущей работе.

1.2. Пример реализации: *Safe, Remote-Access Swarm Robotics Research on the Robotarium*

С увеличением интереса к коллективным роботическим системам (swarm robotics) растёт необходимость экспериментальной проверки алгоритмов вне симуляции. Однако создание собственной инфраструктуры с множеством роботов, системой зарядки, управления и мониторинга — дорого и требует значительных усилий. В работе [Pickem et al TODO] (<https://arxiv.labs.arxiv.org/html/1604.00640>). описана платформа Robotarium — доступная удалённо многоагентная роботическая лаборатория, специально разработанная для исследований и образовательных задач.

Основной вызов: обеспечить максимальную гибкость для пользователей (отправка собственного кода, экспериментальные алгоритмы) и одновременно гарантировать безопасность оборудования (коллизии, неправильная эксплуатация).

Авторы выделяют следующие ключевые характеристики, которым должна соответствовать платформа удалённого доступа в контексте коллективных роботов:

1. Простота и открытость дизайна, чтобы лабораторию могли воспроизводить исследователи и студенты;
2. Интуитивный интерфейс и лёгкость сбора данных;
3. Плавный переход от симуляции к физическому исполнению алгоритмов;
4. Минимальные расходы на поддержку большого числа роботов;
5. Расширяемость (добавление большего числа роботов и разных типов);
6. Встроенные меры безопасности и защиты оборудования.

Эти требования отражаются в архитектуре Robotarium и подробно описываются далее.

1.2.1. Архитектура Robotarium

Платформа делится на две основные части: инфраструктуру непосредственно лаборатории (роботы, арена, система отслеживания, зарядка) и веб-интерфейс + бекэнд для удалённых пользователей.

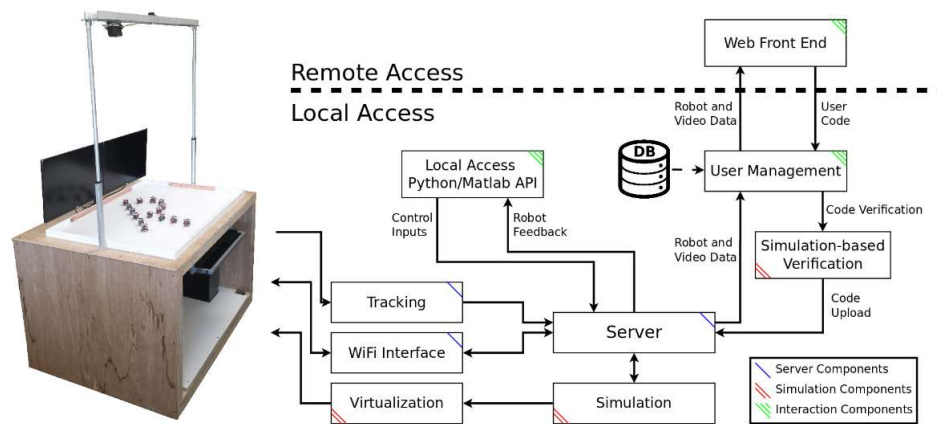


Рисунок 1.7 Обзор архитектуры Robotarium

Инфраструктура включает глобальную систему позиционного отслеживания, автоматическую зарядку роботов, беспроводное программирование, управление задачами. Пользовательский интерфейс рассчитан на загрузку пользовательского кода, проверку его (симуляция), отдачу команд и получение логов/видео.

Аппаратная часть:

- Роботы: платформа использует специально разработанных мини-роботов GRITSBot, дифференциального привода, с возможностью модификации аппаратной части.
- Система отслеживания: глобальная позиция всех роботов вычисляется с помощью камеры сверху и меток ArUco.
- Автоматическая зарядка: роботы оснащены соединителями, подключающимися к магнитным зарядным полосам арены, что позволяет им самостоятельно возвращаться на базу.
- Управление беспроводное и перепрограммирование по WiFi.

Программная часть:

- Серверное приложение координирует загрузку пользовательского кода, маршрутизацию команд и данных, логирование и выдачу результатов.
- API пользователя: через Matlab-API или веб-интерфейс пользователь загружает свой алгоритм, который сначала проходит симуляцию, затем проверку (safety-verification) и после — исполнение на роботе.
- Симулятор используется не только для прототипирования, но и для проверки безопасности кода перед запуском на физических роботах.

- Абстракции доступа: пользователям предоставляется несколько уровней управления — от задания скоростей колёс, до управления топологией/поведением группы роботов.

1.2.2. Безопасность: ключевой компонент удалённой платформы

Одним из центральных вкладов работы является разработка механизма обеспечения безопасности многороботной системы при удалённом доступе.

Двойной механизм (симуляция + онлайн):

- Симуляционная верификация: перед выполнением пользовательского кода на физической платформе система проверяет алгоритм на коллизии и другие нежелательные эффекты. Алгоритм получает «safety score», характеризующий вероятность безопасного выполнения.
- Онлайн-барьеры (Barrier Certificates): если проверка показывает риск, при исполнении используется механизм «контроль ближайшего безопасного действия» — пользовательский сигнал минимально корректируется так, чтобы соблюсти безопасное множество состояний.

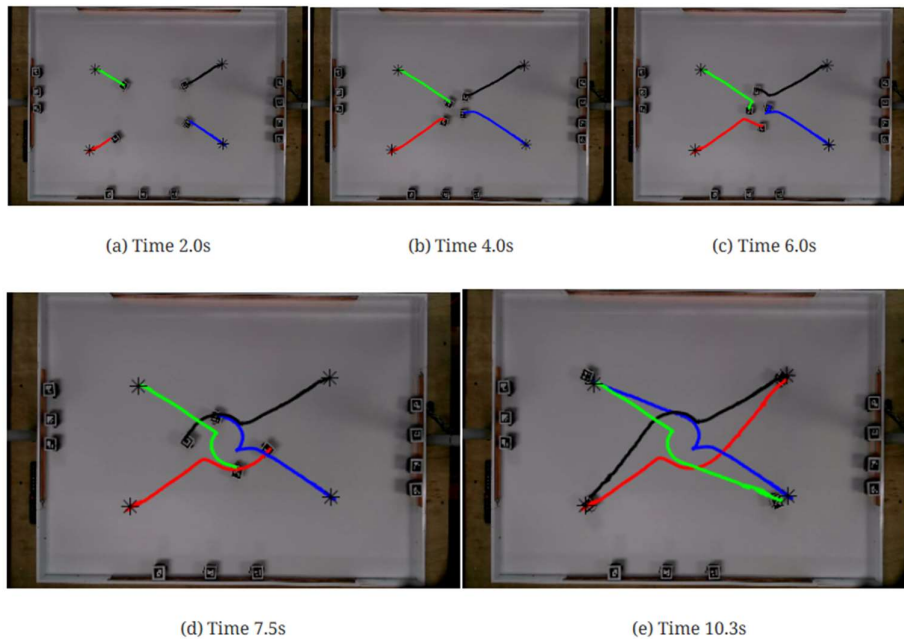


Рисунок 1.8 Демонстрация работы барьеров безопасности

Принцип работы барьеров такой. Для начала формулируется множество безопасных состояний (например, минимальное расстояние между роботами, границы арены). Затем определяется контрольный барьер-функция (control barrier function), которая гарантирует, что, если система внутри безопасного множества, она остаётся в нём. При

каждом такте решается квадратичная программа (QP): минимизируется отклонение от пользовательского управляющего сигнала с сохранением ограничений безопасности. Таким образом — минимально инвазивная безопасность.

1.2.3. Примеры применения и экспериментов

Сходимость (Rendezvous)

Проблема: группа агентов должна сблизиться к одной точке, используя локальную информацию. Код расположен в Matlab-API.

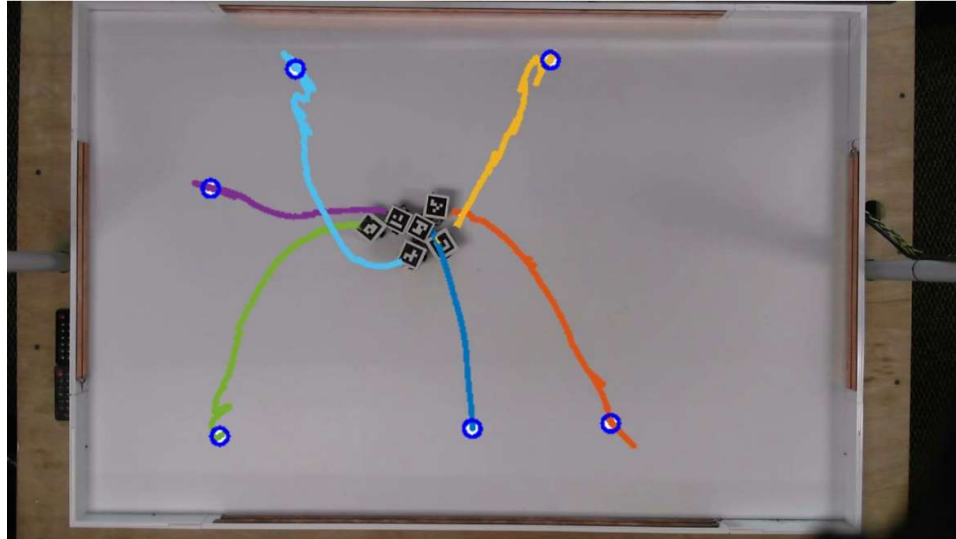


Рисунок 1.9 Траектории агентов при выполнении алгоритма rendezvous

Контроль формирования (Formation Control)

Проблема: агенты должны занять конфигурацию с заданными межагентными расстояниями. Реализовано на Robotarium.

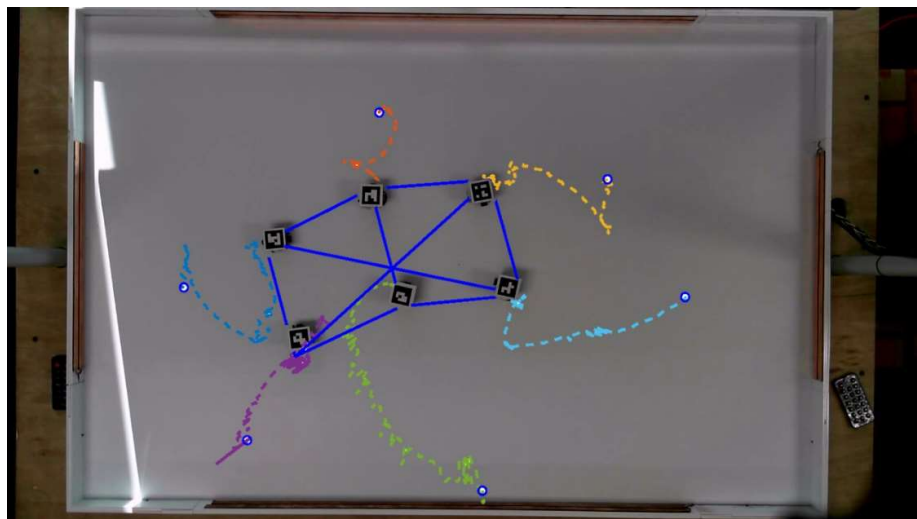


Рисунок 1.10 Траектории и топология связи при формировании

Контроль покрытия (Coverage Control)

Проблема: роботы должны распределиться по области для покрытия плотности, заданной пользователем (например, с сенсорного интерфейса). Реализовано с использованием контроллера распределённой оптимизации и поддержки разных интерфейсов.

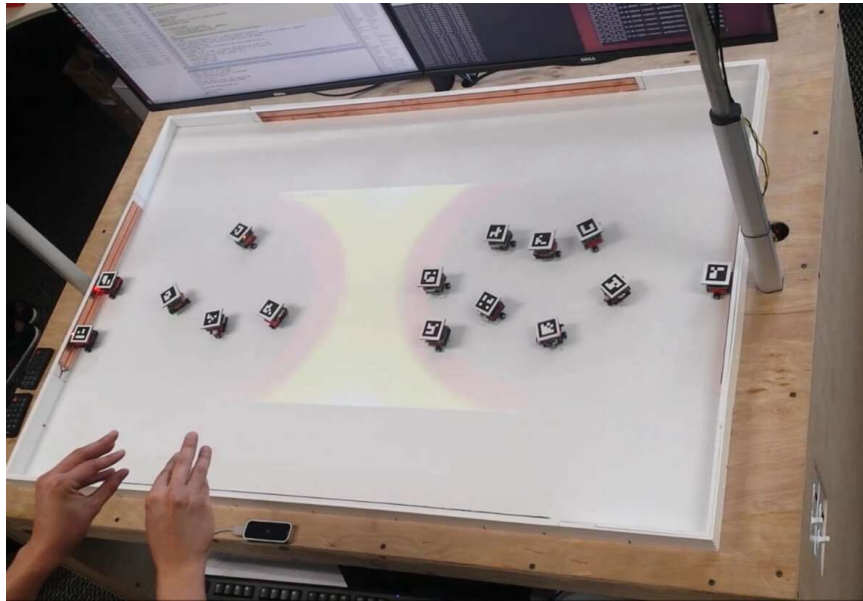


Рисунок 1.11 Эксперимент по покрытию области с использованием сенсорного ввода

1.2.4. Выводы и значение для проекта

Работа Pickem et al. демонстрирует, как можно построить одновременно гибкую и безопасную платформу для удалённого доступа к множеству роботов. Для дипломного проекта она представляет собой важный ориентир, особенно в части:

- проектирования архитектуры с учётом удалённого доступа;
- обеспечения безопасности аппаратуры при выполнении стороннего кода;
- организации симуляции + физического исполнения;
- поддержки мульти-агентных сценариев.

1.3. Пример реализации: A Remote Lab for Experiments with a Team of Mobile Robots

В работе [Casini et al., 2014 TODO] (<https://www.mdpi.com/1424-8220/14/9/16486>) описана удалённая лаборатория для экспериментов с группой мобильных роботов, разработанная в Университете Сиены (Италия) и встроенная в существующую платформу дистанционного обучения Automatic Control Telelab (ACT). В отличие от рассмотренных ранее систем, акцент здесь сделан на образовательном и исследовательском применении в области теории управления: пользователь описывает закон управления группой роботов

на языке Matlab и немедленно проверяет его на реальном стенде через браузер. Ниже приведено описание архитектуры этой системы и сценариев её использования.

1.3.1. Аппаратная архитектура

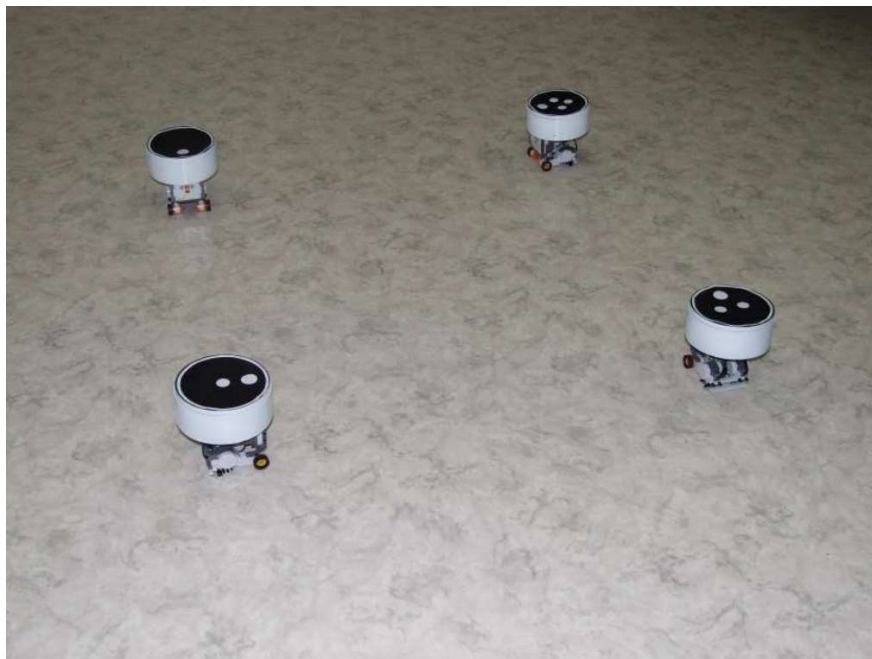


Рисунок 1.12 LEGO NXT mobile robots

Стенд (Рисунок 1.12) состоит из четырёх одинаковых мобильных роботов, передвигающихся в рабочем пространстве размером 4.5×3 м. Роботы собраны на базе конструктора LEGO Mindstorms NXT — платформы, ставшей фактическим стандартом в образовательной робототехнике. Каждый робот имеет дифференциальный привод (два независимо управляемых колеса и стальной шарик-опору) и описывается кинематической моделью одноколёсника (unicycle): состояние робота задаётся позицией и ориентацией $[x, y, \theta]$, а управление — линейной и угловой скоростью.

Ключевые элементы аппаратной части:

1. **Вычислительное ядро робота**— «кубик» NXT с микропроцессором, который обеспечивает вычисления, управление моторами и связь по Bluetooth. На NXT выполняется низкоуровневая программа на языке NXC (Not eXactly C), реализующая ПИ-регулятор скорости колёс по опорному сигналу, получаемому по Bluetooth.
2. **Система визуального позиционирования** — две широкоугольные камеры, закреплённые на потолке лаборатории. По специальным маркерам на верхней

части роботов система вычисляет глобальные позицию и ориентацию каждой машины. Две камеры используются для расширения поля зрения ценой большей вычислительной нагрузки; изображения корректируются от бочкообразной дисторсии и сшиваются в единую систему координат.

3. **Автоматическая зарядка** — по завершении эксперимента специальная процедура приводит роботов в «бокс», где их литиевые батареи заряжаются без участия человека через две металлические пластины на днище робота. Это обеспечивает круглосуточную доступность стенда.

1.3.2. Программная архитектура

Система построена по клиент-серверной схеме, причём серверная часть полностью реализована как Matlab-функция, выполняющая следующие задачи:

1. **Детектирование роботов (Robot Detection).** Кадры с двух камер захватываются средствами Matlab Image Acquisition Toolbox и обрабатываются для выделения маркеров (Image Processing Toolbox). После калибровки и коррекции дисторсии изображения сшиваются, и через однородные преобразования вычисляются глобальные координаты роботов. Сжатая версия изображения используется для онлайн-видеотрансляции.
2. **Управление роботами (Robot Control).** Пользовательский контроллер — это Matlab-функция, вызываемая на каждом такте. На вход она получает позы всех роботов и структуру с состоянием эксперимента, на выходе возвращает желаемые линейную и угловую скорости для каждого робота. Использование Matlab позволяет задействовать богатый набор готовых функций и снижает порог входа для студентов.
3. **Связь с роботами (Communication).** На каждом такте вычисленные скорости передаются роботам по Bluetooth; роботы, в свою очередь, могут сообщать серверу о своём состоянии (например, уровне заряда).
4. **ТСР-соединение с интерфейсом.** Взаимодействие с пользователем организовано через ТСР-соединение между Matlab-функцией сервера и Java-апплетом, реализующим графический интерфейс. Пользователь запускает и останавливает эксперимент, а сервер транслирует апплету позы роботов для визуализации в реальном времени.

5. **Хранение данных (Data Storage).** В ходе эксперимента все значимые данные (позы роботов, вычисленные команды) сохраняются и доступны для скачивания по завершении — для офлайн-анализа.
6. **Система безопасности (Safety System).** Эксперимент автоматически останавливается при выходе робота за границы рабочего пространства или при любой ошибке (например, обрыве TCP-соединения); запускается процедура, приводящая роботов в безопасное положение.

Для доступа к лаборатории, навигации по сайту, загрузки контроллеров и скачивания результатов на сервере дополнительно работает стандартный веб-сервер Apache с поддержкой PHP.

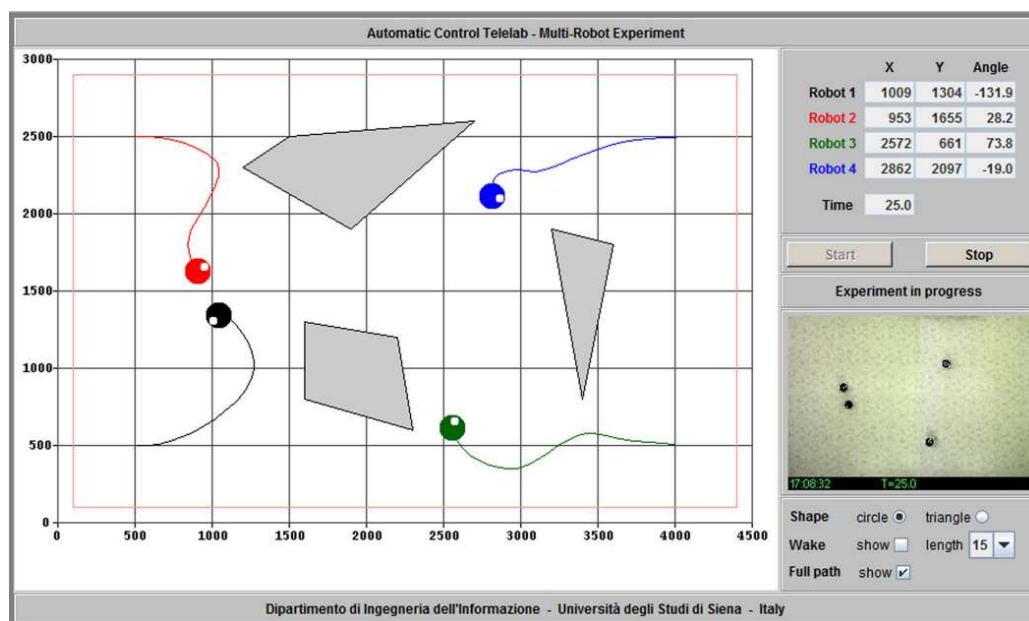


Рисунок 1.13 Пользовательский интерфейс

1.3.3. Сценарий взаимодействия

Для проведения эксперимента пользователю достаточно браузера с поддержкой Java; Matlab нужен только при желании предварительно проверить контроллер в симуляции. С главной страницы пользователь выбирает тип эксперимента — например, движение в пустом пространстве или обход виртуальных препятствий. Затем в окне Control Interface выбирается готовый закон управления либо загружается собственный (на основе предоставляемого шаблона Matlab-функции простого синтаксиса). После выбора контроллера открывается Java-апплет, из которого можно запустить и остановить эксперимент, наблюдать графическое представление и видеопоток.

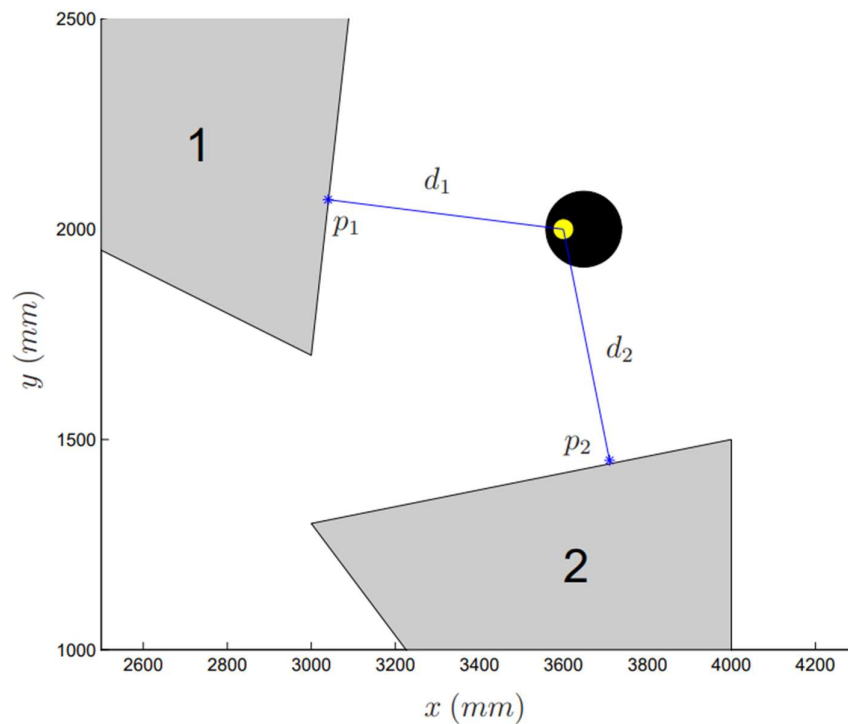


Рисунок 1.14 Виртуальные препятствия

Существенная особенность — поддержка **виртуальных препятствий** (Рисунок 1.14): пользователь задаёт произвольное число полигональных препятствий, а система на каждом такте автоматически вычисляет для каждого робота расстояния до препятствий и до других роботов (рассматривая их как подвижные препятствия), а также карту занятости (occupancy grid). Эти данные передаются в пользовательский контроллер, упрощая реализацию алгоритмов обхода и предотвращения столкновений.

Дополнительно предоставляются два локальных инструмента: **System Simulator** (прогон контроллера в симуляции с графической анимацией до запуска на реальном стенде — для проверки корректности и предсказания траекторий) и **Experiment Player** (воспроизведение записанного эксперимента из скачанных данных).

1.3.4. Примеры экспериментов

Универсальность стенда демонстрируется тремя классами задач:

- **Игры преследователь–убегающий (Pursuer–Evader).** Неконкурентный сценарий теории дифференциальных игр: один агент стремится догнать другого. На стенде студентам предлагалось разработать алгоритм убегающего против неизвестного им преследователя; сложность регулируется соотношением скоростей. Сравнение реальных траекторий с симуляцией показало как случаи хорошего совпадения, так

и случаи существенного расхождения (для контроллеров со сложной динамикой на основе потенциальных полей) — что наглядно иллюстрирует ценность испытаний на реальном оборудовании.

- **Обход препятствий (Collision Avoidance).** На базе метода искусственных потенциальных полей с учётом неголономных ограничений робота. Возможность задавать виртуальные препятствия позволяет безопасно тестировать алгоритмы планирования без риска повредить технику.
- **Координация движения группы (Multi-Agent Motion Coordination).** Поддержка как централизованных, так и децентрализованных законов управления. Продемонстрированы алгоритмы коллективного кругового движения и циклического преследования. Хотя роботы физически не несут внешних сенсоров и не общаются напрямую, система **программно моделирует** наличие виртуальных датчиков (лазерные дальномеры, сонары с заданной точностью) и каналов межагентной связи с ограниченным радиусом — на основе позиций, получаемых от системы технического зрения. Это позволяет реалистично испытывать децентрализованные стратегии.

1.3.5. Выводы и значение для проекта

Работа Casini et al. показывает, как при минимальных затратах построить полнофункциональный многороботный стенд, пригодный одновременно для обучения и исследований. Для настоящего проекта особенно значимы следующие идеи:

- **Безопасность как обязательный модуль** — автоматический останов при выходе за рабочую зону и при сбоях связи (перекликается с механизмом аварийной остановки и контролем состояния, заложенными в разрабатываемую систему).
- **Симуляция перед исполнением** — проверка пользовательского кода до запуска на реальном оборудовании (сравнимо с подходом Robotarium).
- **Программное моделирование сенсоров и каналов связи** поверх точной информации о позах — гибкий способ расширять возможности «тонких» в плане бортовой электроники роботов за счёт вычислений на сервере.
- **Централизация вычислений:** вся логика управления выполняется на сервере, а на борту робота остаётся лишь низкоуровневый регулятор — архитектурное решение, близкое к выбранному в данной работе.

В то же время у системы есть ограничения, которые учитываются при проектировании собственной лаборатории: жёсткая привязка к среде Matlab и Java-апплету (что снижает кроссплатформенность и уже отмечается авторами как проблема в

связи с прекращением поддержки Java), ориентация на единственный тип роботов (LEGO NXT), а также отсутствие развитой системы многопользовательского доступа с авторизацией и очередью.

1.4. Сводное сравнение рассмотренных систем

Ниже сведены ключевые характеристики трёх рассмотренных систем. Сравнение проводится по признакам, существенным для разрабатываемой виртуальной лаборатории и сформулированным далее в требованиях (раздел 0).

Характеристика	rosbridge (Lee, 2012)	Robotarium (Pickem et al., 2017)	Remote Lab Siena (Casini et al., 2014)
Назначение	Веб-приложения для роботов	Удалённая лаборатория роя роботов	Удалённая лаборатория мобильных роботов
Гетерогенность	Любой ROS-робот	Однотипные роботы	Один тип (LEGO NXT)
Middleware / основа	ROS + rosbridge	Собственный сервер + Matlab API	Matlab-сервер + Java-апплет
Протокол клиент / сервер	WebSocket (JSON)	Загрузка кода (Matlab / веб)	TCP (Matlab + Java)
Клиентский интерфейс	Браузер (rosjs, JS)	Браузер + Matlab API	Браузер (Java-апплет) + Matlab
Видеопоток	Да (mjpeg_server, MJPEG)	Да (логи/видео)	Да (сжатые кадры с камер)
Где исполняется логика	Сервер (ROS)	Сервер (загруженный код)	Сервер (Matlab-функция)
Бортовые вычисления	Зависит от робота	Минимальные	Только ПИ-регулятор (NXC)
Позиционирование	—	Камера сверху + ArUco	2 потолочные камеры + маркеры
Многопользовательский доступ	Не рассмотрен	Очередь / расписание	Ограниченный
Безопасность	Не рассмотрена	Barrier certificates + симуляц. верификация	Стоп при выходе за зону / сбое связи
Симуляция перед запуском	—	Да (обязательная верификация)	Да (System Simulator)
Автозарядка	—	Да	Да
Экспорт данных	—	Логи	.mat-файл для офлайн-анализа
Открытость / стандарты	Высокая (ROS, открытый стек)	Частичная	Низкая (Matlab/Java)
Ключевой вклад	Простой доступ к ROS	Безопасность роя при стороннем коде	Лёгкий старт многороботных экспериментов

Выводы из сравнения. Каждая из систем закрывает часть требований к удалённой лаборатории, но ни одна не покрывает их целиком в нужном для данной работы сочетании. От rosbridge берётся идея лёгкого открытого протокола поверх WebSocket и

использование ROS как унифицирующего middleware; от Robotarium — приоритет безопасности и многопользовательского доступа с очередью; от Remote Lab Siena — централизация вычислений с «тонким» бортовым уровнем и обязательный модуль безопасного останова. При этом во всех трёх системах слабо проработана либо гетерогенность роботов, либо многопользовательский режим, либо открытость интерфейса — что и формирует нишу для разрабатываемой системы (см. требования в разделе 1.5).

1.5. Требования к системе управления группой роботов

TODO ПЕРЕФОРМУЛИРОВАТЬ

На основе анализа существующих решений можно сформулировать ключевые требования к виртуальной лаборатории управления группой роботов:

1. **Доступность и кроссплатформенность.** Интерфейс управления должен быть веб-ориентированным, доступным из любого браузера без установки дополнительного ПО. Это позволяет операторам подключаться с разных устройств (компьютеров, планшетов, смартфонов) и платформ (Windows, Linux, macOS, мобильные ОС).
2. **Поддержка нескольких пользователей и расписание.** Система должна предусматривать авторизацию пользователей, систему бронирования времени и очередь доступа к роботам. Это необходимо для согласованного совместного пользования лабораторией и предотвращения конфликтов при одновременных запросах. В частности, как показывают прототипы удалённых лабораторий, пользователь отправляет программу/задачу через веб-интерфейс и дожидается выполнения в назначенное время.
3. **Унифицированная работа с разнородными роботами.** Система должна обеспечивать «абстракцию» для разных типов роботов (разные модели и носители). Например, использование промежуточных оберток вроде ROS позволяет единообразно отправлять команды и получать данные с гетерогенной группы машин. Такое требование предполагает открытые протоколы (WebSocket, MQTT, HTTP/REST и т.п.) и возможность легко добавлять новые устройства.
4. **Реальное время и визуальная обратная связь.** Веб-интерфейс должен обеспечивать видеотрансляцию с камер роботов и/или арены в реальном времени, а также отображать телеметрические данные и состояние роботов (координаты, батарею, нагрузки). Как правило, для телеметрии используют

графики, топики ROS или визуализации на карте. Наличие своевременной обратной связи необходимо для корректного управления и оценки результатов эксперимента.

5. **Управление роботами.** Интерфейс должен предоставлять как минимум средства ручного управления (джойстик, клавиши, консоль команд), так и возможность загружать автономные алгоритмы (скрипты, узлы ROS/программы пользователя). В удалённых лабораториях это реализуется, например, загрузкой MATLAB-кода или ROS-пакетов пользователем. Также полезны «кнопки» быстрого переключения между режимами manual/autonomous и предустановленные шаблоны задач.
6. **Безопасность и надёжность.** Система управления обязана иметь защитные механизмы: аварийную остановку (E-stop) роботов по требованию оператора, ограничения скорости и зоны работы. Должны реализовываться проверки на коллизии и контролироваться состояние оборудования (например, уровень батареи). Логирование сессий и ошибок, а также уведомления об авариях повышают надёжность и позволяют анализировать сбои.
7. **Масштабируемость и модульность.** Архитектура должна быть модульной, чтобы легко расширяться (добавлять новых роботов, датчики, интерфейсы). Применение открытых стандартов и API (WebSockets, ROSBridge, gRPC, GraphQL и т.п.) позволит интегрировать новые компоненты без глобальной переработки системы. Кроме того, важно чтобы серверная часть могла горизонтально масштабироваться (обслуживать больше пользователей и роботов при необходимости).
8. **Удобство интерфейса и адаптивность.** Веб-интерфейс должен быть интуитивным и наглядным: индикация состояния роботов, понятная навигация, настройка представления (например, выбор окон с камерами или дашбордов с данными). Желательно предусмотреть гибкую конфигурацию интерфейса (сохранение пользовательских макетов, подбор используемых модулей), как в современных GUI для робототехники.
9. **Совместимость и открытость.** По возможности система должна опираться на открытые решения и свободные библиотеки (ROS, RobotWebTools, YARP, WebRTC). Это упростит разработку и даст доступ к сообществу разработчиков. При этом нужны четко описанные API и документация, чтобы сторонние разработчики могли расширять платформу.

10. **Мониторинг и отчётность.** Необходимо иметь средства записи (логов, видео) и анализа работы роботов после эксперимента. Автоматический сбор данных и экспорт результатов (графики, отчёты) облегчают научную работу и обучение. Такая функциональность присутствует во многих учебных лабораториях: результаты экспериментов пользователи могут скачать для дальнейшей обработки.

Эти требования сформированы на основе анализа существующих систем. Они отражают ключевые функции удалённых робототехнических лабораторий и платформ: веб-доступность, поддержку мультироботовых/многопользовательских сценариев, гибкость по отношению к типам роботов, а также обеспечение безопасности и информативности управления. В дальнейшем эти требования лягут в основу разработки конкретного ПО для управления группой трёхколёсных мобильных роботов (и других механизмов по единому принципу), с учётом используемых ROS/YARP-технологий и открытых стандартов.

TODO – ОФОРМИТЬ ИСТОЧНИКИ

build/paper.dvi https://cs.brown.edu/media/filer_public/b0/03/b0037d43-76b6-4729-8c1e-635f120e5192/lee.pdf

Frontiers | Connecting YARP to the Web with Yarp.js
<https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2017.00067/full>

A Remote Lab for Experiments with a Team of Mobile Robots
<https://www.mdpi.com/1424-8220/14/9/16486>

InDro Controller - InDro Robotics <https://indrorobotics.ca/indro-controller/>
<https://arxiv.org/html/2406.02210v1>

inetcamera11.dvi <https://vlsicad.ucsd.edu/Publications/Conferences/46/c46.pdf>
https://www.researchgate.net/publication/379320619_Open_remote_web_lab_for_learning_robotics_and_ROS_with_physical_and_simulated_robots_in_an_authentic_developer_environment

Safe, Remote-Access Swarm Robotics Research on the Robotarium
<https://arxiv.labs.arxiv.org/html/1604.00640>

<https://www.ijee.ie/articles/Vol19-3/IJEE1408.pdf>